

Atividade 02 – Compilador CI (Constantes Inteiras)

Andrei Formiga

November 12, 2025

Sumário

1. Atividade 2 - Compilador CI (Constantes Inteiras)	3
1.1. A linguagem CI (Constantes Inteiras)	3
1.2. O compilador	3
1.3. A estrutura de um compilador	5
1.4. O que deve ser entregue	6
1.5. Próximas Etapas	7

1. Atividade 2 - Compilador CI (Constantes Inteiras)

Nesta atividade o objetivo é criar um compilador completo para uma linguagem extremamente simples. O propósito é mostrar as etapas de um compilador no contexto mais simples possível. Nas próximas atividades a complexidade da linguagem e do compilador vai aumentar gradualmente.

1.1. A linguagem CI (Constantes Inteiras)

Um programa na linguagem CI é apenas uma constante inteira, formada por um ou mais dígitos. Uma gramática para essa linguagem pode ser especificada da seguinte forma:

```
<programa> ::= <literal-inteiro>
<literal-inteiro> ::= <digito>+
<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Por exemplo, este é um *programa* na linguagem CI:

42

A linguagem CI é extremamente limitada como linguagem de programação; a única capacidade dos programas em CI é imprimir diferentes constantes inteiras na tela. Mesmo assim, já é possível aprender ideias mais gerais sobre compiladores mesmo nessa simplicidade.

1.2. O compilador

Os compiladores que veremos na disciplina seguem a mesma estrutura: a partir do código-fonte na entrada, o compilador vai gerar um programa equivalente em linguagem *assembly*. Este programa em assembly precisa ser montado (por um *assembler*, ou *montador* em português) e ligado por um *linker* para se tornar um executável (Figura 1).

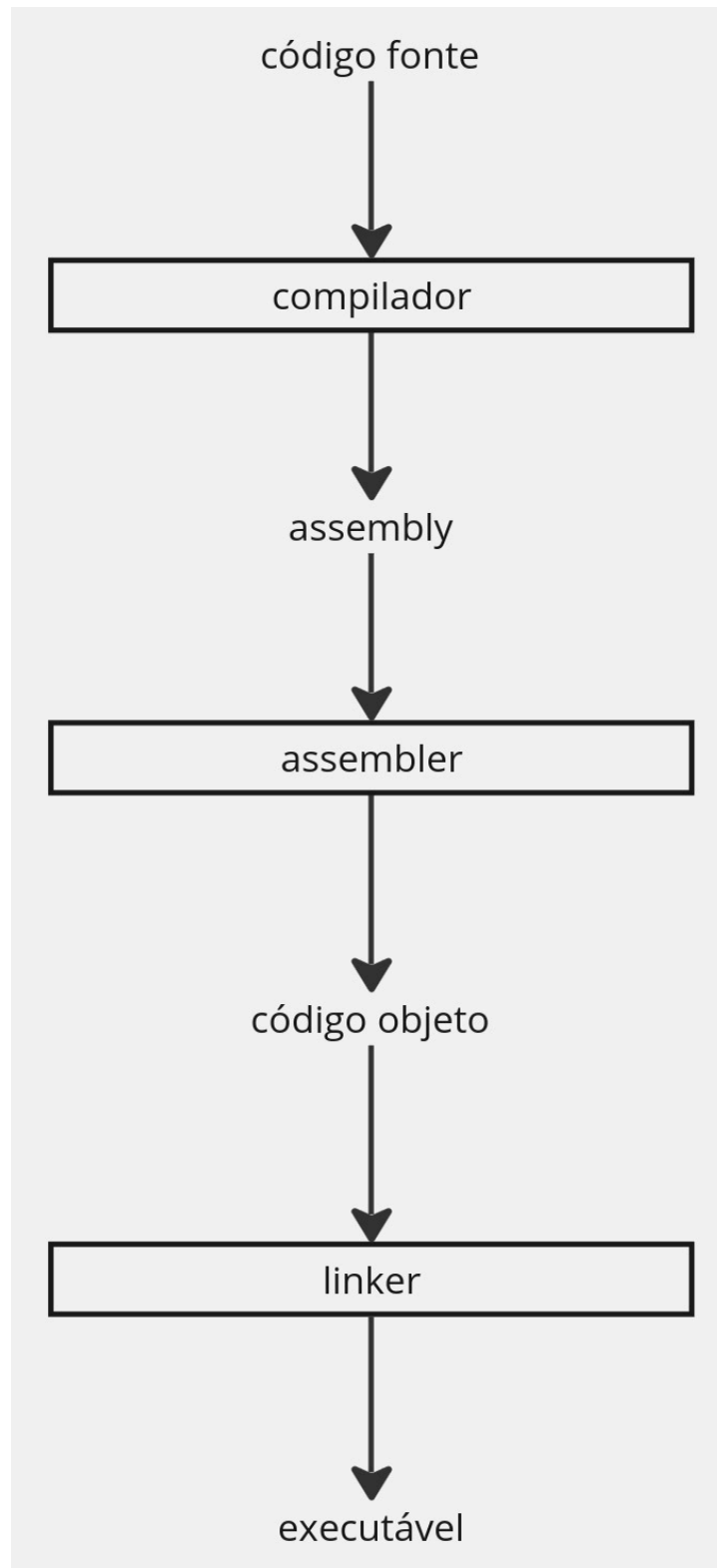


Figura 1: Processo para gerar um executável

O compilador desta atividade deve ler o programa em um arquivo e gerar um programa *assembly* na saída que imprime o resultado do programa. A maior parte deste programa vai seguir um modelo que veremos em seguida; o que o compilador CI precisa gerar é apenas uma linha desse programa *assembly*: uma instrução que coloca o resultado do programa no registrador RAX. Por exemplo, se o arquivo `p1.ci` contém a constante 42, o compilador deve gerar a seguinte linha de código *assembly*:

```
mov $42, %rax
```

Os exemplos na disciplina usarão a linguagem assembly para a arquitetura x86-64 e a sintaxe do GNU Assembler (gas), executando em um sistema operacional de kernel Linux. Cada grupo pode escolher gerar código para outras arquiteturas, outros sistemas operacionais, e/ou mudar o assembler utilizado.

A linha gerada pelo compilador deve ser incluída em um modelo de arquivo assembly que imprime o conteúdo do registrador RAX:

```
#
# modelo de saída para o compilador
#

.section .text
.globl _start

_start:
## saída do compilador deve ser inserida aqui

call imprime_num
call sair

.include "runtime.s"
```

A linha gerada pelo compilador deve ser colocada no local marcado com o comentário, logo após o rótulo `_start`. Podemos ver que esse modelo chama o procedimento `imprime_num` (que imprime na tela o valor do registrador RAX como um inteiro) e depois o procedimento `sair` para sinalizar ao sistema operacional que o programa acabou. O arquivo `runtime.s` que é incluído ao final do modelo contém as definições desses procedimentos.

Por enquanto não é necessário entender o modelo utilizado nem o arquivo `runtime.s`. Faremos uma revisão sobre assembly em breve.

O arquivo assembly gerado pelo compilador (contendo a linha de resultado inserida no modelo acima) deve ser montado e *linkado*. Se o arquivo gerado pelo compilador foi chamado de `p1.s`, os comandos para montar e *linkar* são:

```
as --64 -o p1.o p1.s
ld -o p1 p1.o
```

Isso gera o executável de nome `p1` que, ao ser executado, vai imprimir a constante inteira que estava no arquivo fonte original na tela.

1.3. A estrutura de um compilador

O compilador CI é muito simples, mas já serve para demonstrar a estrutura de um compilador no geral.

Se pensarmos sobre como deve ser o algoritmo do compilador CI, vemos que pelo menos os seguintes passos são necessários:

1. Ler o arquivo de entrada

2. Usar a constante lida no arquivo de entrada e o modelo da saída para gerar o programa assembly resultante
3. Escrever o programa de saída no arquivo de saída

Esse algoritmo não leva em consideração o que acontece caso o programa de entrada tenha um erro de sintaxe; para a linguagem CI, um erro de sintaxe significa que o arquivo de entrada não contém uma constante inteira correta (por exemplo, inclui letras).

Isso demonstra a necessidade do tratamento de erros. Se o programa na entrada não contém uma constante inteira correta, não será possível gerar o arquivo de saída.

(Outro tipo de erro que pode ocorrer é a constante no programa de entrada ser grande demais para caber em um registrador de 64 bits. A verificação desse tipo de erro é um pouco mais difícil e é opcional para esta atividade).

Vamos incluir mais um passo no algoritmo:

1. Ler o arquivo de entrada
2. Verificar se o arquivo de entrada contém uma constante inteira
3. Usar a constante lida no arquivo de entrada e o modelo da saída para gerar o programa assembly resultante
4. Escrever o programa de saída no arquivo de saída

Neste compilador muito simples vemos que um compilador pode ser dividido em duas etapas principais: uma etapa de *análise* e outra etapa de *síntese* ou *geração*. A etapa de análise é responsável por coletar as informações necessárias do programa de entrada, ao mesmo tempo verificando pela presença de erros na entrada. A síntese gera o programa de saída, usando as informações coletadas durante a análise.

A verificação de sintaxe no passo 2 do algoritmo acima é a etapa de análise do compilador CI. O passo 3 é a etapa de síntese ou geração de código.

Todos os compiladores que veremos nesta disciplina seguirão essa estrutura de análise e síntese.

1.4. O que deve ser entregue

Nesta atividade cada grupo deve enviar o programa do compilador para a linguagem CI. O compilador deve receber, na entrada, o nome do arquivo que contém o programa a ser compilado. O compilador deve produzir, na saída, um arquivo assembly que segue o modelo apresentado neste guia, mas que inclui a linha com a instrução `mov` que coloca o valor da constante no registrador RAX. Este arquivo assembly deve poder ser montado e linkado corretamente para gerar um executável que imprime a constante na tela (desde que o arquivo `runtime.s`, fornecido, esteja presente no mesmo diretório).

O grupo deve enviar o código-fonte do compilador e, opcionalmente, um `Dockerfile` que permita executar o compilador em um contêiner. O projeto enviado também deve incluir um arquivo de documentação que diz claramente como compilar o compilador (se necessário) e como executar o compilador para algum arquivo de entrada. Também é interessante incluir pelo menos dois testes: um com um programa correto e um teste com um erro de sintaxe.

1.5. Próximas Etapas

Nas próximas etapas, o compilador passará a traduzir uma linguagem de expressões aritméticas. O compilador vai gerar código que calcula o resultado da expressão e coloca esse resultado no registrador RAX. A partir daí, o modelo de código *assembly* visto neste projeto será usado sem alteração para imprimir o resultado.